

FICHE DE PROCÉDURE

Gestion du Trafic Aérien

Infrastructure, Déploiement et Fonctionnement

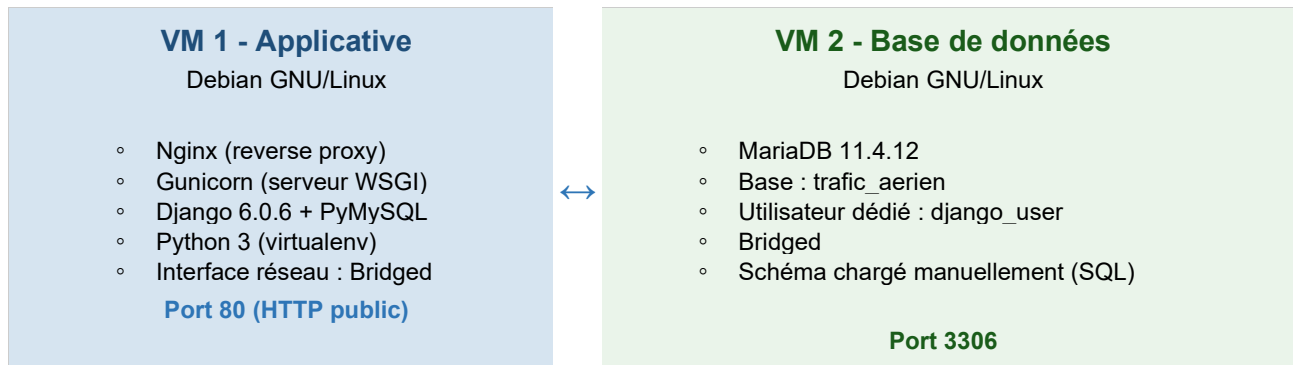
Champ	Valeur
Projet	SAE 2.03 - Application Web Django
Formation	BUT Réseaux & Télécommunications
Groupe	Sujet 7
Équipe	Adil - Lucas - Adel
Dépôt GitHub	github.com/Patpat200/Gestion-Trafic-Aerien-Django
Date de rédaction	Juin 2026
Version	1.0 - Production

Ce document décrit l'infrastructure matérielle et logicielle mise en place pour le projet, le fonctionnement de l'application, ainsi que la procédure complète de déploiement sur les deux machines virtuelles prévues par l'architecture cible.

1. Infrastructure matérielle et logicielle

1.1 Architecture générale

Le projet repose sur une architecture à deux machines virtuelles (VM) hébergées sous VirtualBox. Cette séparation respecte le principe de séparation des couches applicative et données, et reflète les bonnes pratiques en production.



1.2 Configuration réseau VirtualBox

Chaque VM dispose de deux interfaces réseau pour répondre à des besoins distincts :

Interface	Type	VM concernée	Rôle
enp0s3	Bridged	VM 1 Serveur Web	Accès Internet en DHCP, Nginx et Gunicorn installé dessus
enp0s3	Bridged	VM 2 BDD	Accès Internet en DHCP, base de donnée avec MariaDB

1.3 Composants logiciels

Logiciel	Version	Rôle
Python	3.11+	Langage principal du backend
Django	6.06	Framework web MVC
PyMySQL	1.2.0	Driver Python pour MariaDB (pur Python)
Gunicorn	26.0	Serveur WSGI de production
Nginx	1.3.0	Reverse proxy HTTP, gestion des fichiers statiques
MariaDB	11.4.10	SGBD relationnel (base de données)
Bootstrap 5	5.3	Framework CSS front-end
Git / GitHub	-	Versionnement et dépôt du code source

2. Fonctionnement du projet

2.1 Schéma de la base de données

Le schéma est défini manuellement via des fichiers SQL (schema.sql et donnees.sql). Ceci est une contrainte imposée par le cahier des charges.

Table	Champs principaux	Relations
aéroport	id, nom, pays	-
piste	numero, aéroport_id, longueur	FK vers aéroport
compagnie	id, nom, description, pays	-
type_avion	id, marque, modele, description, longueur_piste_min	-
avion	id, nom, compagnie_id, type_id	FK vers compagnie, type_avion
vol	id, avion_id, pilote, aéroport_dep_id, date_dep, aéroport_arr_id, date_arr	FK vers avion, aéroport (x2)

2.2 Règles métier implémentées

Règle des 10 minutes (disponibilité des pistes)

Lors de l'ajout d'un vol, le système vérifie qu'une piste compatible est disponible à l'aéroport d'arrivée à l'heure prévue. Une piste est considérée occupée pendant 10 minutes après l'atterrissage d'un avion. Si aucune piste n'est libre, l'application propose automatiquement le premier créneau disponible.

Logique de vérification

1. Récupérer toutes les pistes de l'aéroport d'arrivée dont la longueur $\geq$ longueur minimale du type d'avion
2. Pour chaque piste : vérifier si un vol existant arrive dans la fenêtre $[date_arr - 10min, date_arr + 10min]$
3. Si une piste est libre $\rightarrow$ vol accepté
4. Si toutes les pistes sont occupées $\rightarrow$ proposer $date_arr = \text{heure_libération_piste_la_plus_proche} + 1min$

Import CSV de vols en masse

Une page dédiée permet d'importer un fichier CSV contenant plusieurs vols depuis un aéroport. Le format attendu est décrit en préambule de la page d'import :

Format CSV attendu

| # En-tête obligatoire (1ère ligne) :

avion_id,pilote,aéroport_depart_id,date_depart,aéroport_arrivee_id,date_arrivee

| # Exemple de ligne :

3,Jean Dupont,1,2026-06-15 10:30,2,2026-06-15 13:45

| # Format des dates : YYYY-MM-DD HH:MM

Fiche des vols (export/impression)

L'application permet de générer une fiche imprimable des vols au départ ou à destination d'un aéroport pour une date ou période données. Un filtre par aéroport, direction et plage de dates est disponible. La page intègre un bouton d'impression directe via le navigateur (CSS @media print).

2.3 CRUD implémenté

Un CRUD complet (Create, Read, Update, Delete) est disponible pour les 6 entités :

- Aéroports
- Pistes d'atterrissage
- Compagnies aériennes
- Types d'avions (avec longueur de piste minimale)
- Avions
- Vols (avec vérification automatique de la disponibilité des pistes)

2.4 Tableau de bord

Une page d'accueil (dashboard) affiche en temps réel les compteurs de la base de données : nombre d'aéroports, de pistes, de compagnies, de types d'avions, d'avions et de vols enregistrés. Les données sont lues directement depuis la base à chaque chargement de page.

3. Procédure de déploiement

3.1 Prérequis

Avant de commencer, s'assurer que :

- VirtualBox ou VMWare est installé sur la machine hôte
- Deux VMs Debian sont créées et configurées avec les interfaces réseau décrites en §1.2
- Avoir le Repository github du projet

3.2 Configuration de la VM 2 - Base de données (MariaDB)

Étape 1 - Installation de MariaDB

```
$ apt update && apt install -y mariadb-server
$ systemctl enable mariadb && systemctl start mariadb
$ mysql_secure_installation
```

Étape 2 - Création de la base et de l'utilisateur

```
$ mysql -u root -p
CREATE DATABASE trafic_aerien CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'django_user'@'%' IDENTIFIED BY 'votre_mot_de_passe';
GRANT ALL PRIVILEGES ON trafic_aerien.* TO 'django_user'@'%';
FLUSH PRIVILEGES;
```

Étape 3 - Chargement du schéma et des données initiales

```
$ mysql -u django_user -p trafic_aerien < schema.sql
$ mysql -u django_user -p trafic_aerien < donnees.sql
```

Étape 4 - Autoriser les connexions distantes

Dans /etc/mysql/mariadb.conf.d/50-server.cnf, modifier :

```
bind-address = 0.0.0.0
$ systemctl restart mariadb
```

3.3 Configuration de la VM 1 - Application (Django + Nginx + Gunicorn)

Étape 1 - Installation des dépendances système

```
$ apt update && apt install -y python3 python3-venv python3-pip nginx git
```

Étape 2 - Récupération du code source

```
$ git clone https://github.com/Patpat200/Gestion-Trafic-Aerien-Django.git
/var/www/gta
$ cd /var/www/gta
```

Étape 3 - Création de l'environnement virtuel et installation des paquets Python

```
$ python3 -m venv venv
$ source venv/bin/activate
$ pip install django pymysql gunicorn
```

Étape 4 - Configuration de settings.py

Éditer le fichier `gta/settings.py` et adapter les paramètres suivants :

Paramètres à configurer dans `settings.py`

```
ALLOWED_HOSTS = ['<IP_VM1>', 'localhost']
```

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'trafic_aerien',  
        'USER': 'django_user',  
        'PASSWORD': 'votre_mot_de_passe',  
        'HOST': '<IP_VM2>',  
        'PORT': '3306',  
        'OPTIONS': {'init_command': "SET sql_mode='STRICT_TRANS_TABLES'"},  
    }  
}
```

Étape 5 - Application des migrations Django (tables internes uniquement)

```
$ source venv/bin/activate  
$ python3 manage.py migrate
```

Note : cette commande ne touche pas aux tables applicatives, déjà créées par `schema.sql`.

Étape 6 - Collecte des fichiers statiques

```
$ python3 manage.py collectstatic --noinput
```

Étape 7 - Configuration de Gunicorn en service systemd

Créer le fichier `/etc/systemd/system/gunicorn.service` :

Contenu de `gunicorn.service`

[Unit]

Description=Gunicorn - Gestion Trafic Aérien

After=network.target

[Service]

User=www-data

WorkingDirectory=/var/www/gta

ExecStart=/var/www/gta/venv/bin/gunicorn --workers 3 --bind unix:/run/gunicorn.sock

gta.wsgi:application

Restart=on-failure

[Install]

WantedBy=multi-user.target

```
$ systemctl daemon-reload  
$ systemctl enable gunicorn && systemctl start gunicorn
```

Étape 8 - Configuration de Nginx

Créer le fichier `/etc/nginx/sites-available/gta` :

Contenu du fichier Nginx

```
server {
    listen 80;
    server_name <IP_VM1>;

    location = /favicon.ico { access_log off; log_not_found off; }

    location /static/ {
        root /var/www/gta;
    }

    location / {
        include proxy_params;
        proxy_pass http://unix:/run/gunicorn.sock;
    }
}
```

```
$ ln -s /etc/nginx/sites-available/gta /etc/nginx/sites-enabled/
$ nginx -t && systemctl restart nginx
```

3.5 Vérification du déploiement

Une fois les étapes terminées, effectuer les vérifications suivantes :

Vérification	Commande	Résultat attendu
Gunicorn actif	systemctl status gunicorn	active (running)
Nginx actif	systemctl status nginx	active (running)
Socket UNIX présent	ls /run/gunicorn.sock	Fichier présent
Application accessible	curl http://<IP_VM1>/	Code HTTP 200
Connexion BDD	python3 manage.py check --database default	System check: no issues

3.6 Dépannage courant

Symptôme	Cause probable	Solution
502 Bad Gateway	Gunicorn ne tourne pas ou socket manquant	systemctl restart gunicorn
ModuleNotFoundError (pymysql)	Gunicorn lancé hors du virtualenv	Vérifier ExecStart dans gunicorn.service (chemin venv/bin/gunicorn)
SyntaxError dans settings.py	Conflit de merge Git sur ALLOWED_HOSTS	Corriger manuellement les marqueurs <<<< et >>>>
Accès refusé MariaDB	IP de VM1 non autorisée ou bind-address incorrect	Vérifier GRANT et bind-address=0.0.0.0 sur VM2
Fichiers statiques 404	collectstatic non exécuté	python3 manage.py collectstatic --noinput

4. Notes de sécurité et bonnes pratiques

- Ne jamais versionner settings.py avec les mots de passe en clair - utiliser des variables d'environnement ou un fichier .env non commité
- Le compte django_user MariaDB ne doit avoir accès qu'à la base trafic_aerien
- Mettre DEBUG = False en production dans settings.py
- Maintenir les paquets Python à jour régulièrement (pip list --outdated)

5. Récapitulatif des livrables du projet

Livrable	Format	Contenu
Fiche de procédure	PDF	Ce document - infrastructure et déploiement
Dossier technique	PDF	Conception, architecture, choix techniques
Portfolio	Site web	Fiche portfolio individuelle avec la partie en anglais
Dépôt GitHub	URL	Code source, schema.sql, donnees.sql, start.sh
Gantt	PNG	Planning des tâches par étudiant
Soutenance	Oral	5 min démo + 10 min questions jury

- Fin du document -